

Monash University

FIT5202 - Data Processing for Big Data (2026 S2)

Assignment 1: Analysing Parking & Congestion Patterns in Melbourne

Due Date: 23:55 Friday 18/Sep/2026 (End of Week 8)

Weight: 15% of the final marks

Background: Navigating Urban Mobility Through Big Data

Modern cities generate continuous streams of operational data, including parking sensor events, traffic counts, and transit feeds, at volumes and velocities that traditional tools struggle to process. This assignment uses Apache Spark to analyse a large-scale historical dataset of on-street parking sensor activity in the City of Melbourne, fused with traffic volume data, to uncover occupancy and congestion patterns that could inform parking policy and traffic management.

The sheer volume, variety, and velocity of this information have pushed urban mobility analysis firmly into the realm of Big Data. Traditional relational databases and standard analytical methods often struggle to scale when processing millions of sensor records. To understand the true underlying causes of traffic congestion and parking shortages, we must look beyond isolated events and analyse complex, overlapping factors, such as how traffic volume correlates with parking turnover and violation rates across diverse geographic zones.

Big Data processing frameworks provide the distributed computing tool necessary to handle this complexity. By leveraging these modern tools, data engineers and analysts can:

- **Process at Scale:** Ingest, clean, and standardise massive, messy datasets across distributed clusters.
- **Execute Complex Data Fusions:** Perform multi-attribute joins to merge disparate data sources (e.g., linking parking occupancy to specific zones and local geographical data) without crashing the system.
- **Uncover Hidden Patterns:** Execute parallel aggregations to identify high-demand "hotspots," temporal congestion trends, and usage profiles.

Objective

In this assignment, you will utilise Apache Spark to analyse a large-scale historical parking and traffic dataset. You will be tasked with parsing raw files, handling missing or invalid data, and writing optimised queries to perform multivariate analysis. By applying appropriate data partitioning and parallel processing strategies, your goal is to transform raw, fragmented sensor and traffic data into actionable insights that could inform public policy, improve road infrastructure, and ultimately improve urban mobility.

The assignment assesses your ability to:

- Prepare and validate a large dataset;
- Use the PySpark DataFrame and SQL APIs;
- Join, group, sort and aggregate distributed data;
- Interpret Spark physical plans and Spark UI evidence;
- Evaluate an optimisation using measured results.

The Dataset

The download link for the dataset can be found on Moodle; it consists of historical sensor records and traffic volume data from the City of Melbourne.

You will find the following files after extracting the zip file:

- 1) parking_sensor_data.csv: Contains on-street parking sensor activity records.
- 2) area.json: Mapping table for parking areas to sensor devices.
- 3) street.json: Mapping table for parking street to sensor devices.
- 4) traffic_count.csv: Traffic sensor measurements in each 10-minute block in a certain area.

Note: Most of the column names are self-explanatory.

Getting Started

- Download your dataset from Moodle.
- Download a template file for submission purposes:
 - **A1_template.ipynb** file in Jupyter Notebook to write your solution. Rename it to the name format (for example: **A1_xxx0000.ipynb**. (xxx0000 is your auth code – **the part before @ in your student email address.**)
- For this assignment, you will use Python 3+ and PySpark 4.x.x. (The environment is provided as a Docker image, the same one you use in labs.)

Part 1: Data Loading and Transformation (10%)

Before analysing Melbourne's parking and traffic patterns, load, clean and validate the raw datasets, then inspect how Spark partitions the large parking sensor data for parallel processing:

1.1 Data Preparation and Loading

- 1) Configure the Spark session to run locally using four cores;
- 2) Assign an appropriate application name to the Spark session;
- 3) Set the session timezone explicitly to 'Australia/Melbourne';
- 4) Define and apply an explicit StructType schema when loading each CSV and JSON dataset.

- 5) **Print** the schema and row count for each DataFrame;
- 6) **Display** the first five rows of each DataFrame to verify successful data loading.

1.2 Data Cleansing and Validation

Real-world datasets are often messy. Before analysing the parking data, you need to clean and validate it so that missing values, incorrect data types and invalid records do not affect the results.

Keep the original DataFrames unchanged and create a **cleaned** version for all DataFrames for the remaining tasks. Some cleaning operations below apply specifically to the parking sensor data:

- 1) Identify the missing-value representations within the dataset and convert them to null. Briefly state which values were treated as missing.
- 2) Parse the *'ArrivalTime'* and *'DepartureTime'* columns and convert them to timestamp format.
- 3) Convert the *"DeviceId"* to an appropriate numeric type.
- 4) Calculate a new *"DurationMinutes"* column using the arrival and departure timestamp. Store the result as a DoubleType value and rounded to two decimal places.
- 5) Identify and report records where the departure time is earlier than the arrival time. Briefly discuss any pattern observed and state how these records will be handled in duration-based queries.
- 6) Explain how records with missing or invalid departure information will be handled in subsequent duration-based queries.
- 7) **Display** and print the schema for *DeviceId*, *ArrivalTime*, *DepartureTime* and *DurationMinutes*.
- 8) **Display** a validation summary table containing the count for each of the following:
 - total parking records;
 - ArrivalTime parsing failures;
 - DepartureTime parsing failures;
 - DeviceId conversion failures;
 - records with missing DepartureTime;
 - records where DepartureTime is earlier than ArrivalTime.

1.3 Partition and Spark UI Evidence

Spark divides data into partitions and processes each partition using a task. In this section, inspect the partitions in the cleaned parking sensor DataFrame created in Section 1.2 and use evidence from the Spark UI to assess whether the work is evenly distributed.

- 1) **Display** the total number of partitions within the cleaned parking sensor DataFrame.
- 2) **Print** the number of records in each partition. Immediately before running this action, set the following Spark Job description:
`sc.setJobDescription(f"A1-P1-{student_auth_code}")`
- 3) Include a set of screenshots from the Spark UI tab that clearly show:
 - a) the job tab with the specified job description;
 - b) the relevant stage and its number of tasks
 - c) the task input size, input records or task-duration distribution.
- 4) Analyse the figures in the screenshot and briefly comment on whether the workload appears evenly distributed across the partitions. If not, why?
- 5) Provide a concise explanation detailing how Spark determines the initial number and approximate size of input partitions when reading from a CSV file.

Part 2. Working with DataFrames (35%)

With the datasets cleaned and validated, you can now investigate how parking activity varies across Melbourne. Use DataFrame functions to answer the following queries:

2.1 Create the time attributes

Parking demand can vary considerably between peak and off-peak periods. Create the time attributes needed to compare these periods.

- 1) From *ArrivalTime*, create columns for: year; month; day of week; arrival hour and arrival minute;.
- 2) With day of week and hour and minutes, create a new column named **isPeak** and set the value to True or False (i.e. True = Peak Hour, False = Non-peak hour).
 - Peak hours are:
 - **Weekdays** 7:00 - 9:00 am (excluding 9:00 am) and 4:00 - 6:30 pm (excluding 6:30 pm).
 - Non-peak hours are times outside these periods.
 - Represent day of week using Spark's convention: 1 = Sunday, 2 = Monday.....
- 3) **Display** five rows containing ArrivalTime, year, month, day of week, arrival hour, arrival minute and isPeak, and print the relevant schema.

2.2 Join the parking data with the area and street information

Join the cleaned parking sensor data with both lookup tables to add the area and street names. (Note: The area and street lookup table is provided in JSON format).

- 1) Select an appropriate join type;
- 2) Report the number of parking records that do not match an area look up row and the number that do not match a street lookup row.
- 3) Identify a suitable join strategy for these joins and justify your choice.
- 4) **Display** the first five joined records with the following columns: *DeviceId*, *AreaId*, *AreaName*, *StreetId*, and *StreetName*.

2.3 Compare parking activity and violation rates

The council would like to compare parking activity and violation patterns during peak and off-peak periods.

For each parking area:

- 1) **PeakDailyAverage**: the average number of valid peak-hour sessions per active weekday.
- 2) **PeakViolationRate**: percentage of valid peak-hour sessions marked as **InViolation**.
- 3) **OffPeakDailyAverage**: the average number of valid off-peak sessions per active day.
- 4) **OffPeakViolationRate**: the percentage of valid off-peak sessions marked as **InViolation**.
- 5) **TotalValidSessions**: total peak and off-peak sessions.
- 6) **Display** the ten areas with the highest *TotalValidSessions*, ordered by *TotalValidSessions* descending. If counts are equal, order by *AreaId* ascending.
- 7) Briefly compare the peak and off-peak results and discuss whether higher parking activity appears to correspond with higher violation rates among these areas.

Note:

- Violation rate = (sessions where *InViolation* is true) / (sessions with a valid *InViolation* value in the same period) * 100%.
- A valid session must have a non-null *AreaId*, and a successfully parsed *ArrivalTime*.
- Calculate each violation rate using sessions with a valid *InViolation* value in the corresponding period.
- A missing *DepartureTime* does not invalidate a session because duration is not used in this task.
- An active day is a date on which the area has at least one valid parking session.

2.4 Identify high-use and long-stay parking bays

Parking planners would like to identify bays with frequent vehicle turnover and bays that are commonly occupied for longer periods.

For each BayId, calculate:

- 1) the total number of valid parking sessions;
- 2) the total number of sessions with a valid duration;
- 3) the average parking duration in minutes, rounded to two decimal places
- 4) the percentage of sessions marked as InViolation, rounded to two decimal places

Identify and **display**:

- 1) the top 5 bays with the highest turnover, order the turnover result by totalSessions descending, use BayID ascending to resolve ties.
- 2) the top 5 bays with the longest average stay, order the longest-stay result by AverageDurationMinutes descending, use BayID ascending to resolve ties.
- 3) For both top-5 results, display BayID, TotalSessions, ValidDurationSessions, AverageDurationMinutes and ViolationRates.

Note:

- A valid session must have a non-null identifier required by the query, a successfully parsed ArrivalTime.
- Define turnover as the total number of valid sessions recorded for a bay during the observation period

2.5 Combine parking activity with traffic volume

The council would like to identify when an area experiences unusually high traffic and parking activity at the same time.

The traffic count dataset is measured by sensors during 10-minute time blocks. A time block begins at 00, 10, 20, 30, 40, or 50 minutes past the hour. For example, times from 00:00:00 inclusive to 00:10:00 exclusive belong to the block beginning at 00:00:00. A traffic record with: 2018, 1, 1, 0, 0 is the first 10-minute block on 1/1/2018, and belongs to the block 2018-01-01 00:00:00 to 2018-01-01 00:09:59.

- For each area-time block:
 - Define *ParkingSessions* as the number of parking sessions beginning within the block.
 - Define *TrafficVolume* as the sum of traffic_count within the block.
- 1) Now, you need to think of a strategy to combine parking activity and traffic volume. Select an appropriate join type and justify your choice.
- 2) Across the matched area-time blocks, define high parking activity and high traffic volume as values greater than their respective averages.

Display a summary containing:

- the average ParkingSessions, average TrafficVolume, number of matched area-time blocks, and the number where both measures are above average.

- 3) **Display** the ten above-average combinations with the highest *ParkingSessions*, display *AreaID*, *AreaName*, *BlockStart*, *ParkingSessions*, *TrafficVolume*, order the result by *ParkingSessions* and *TrafficVolume* in descending order. If values are equal, order by *AreaID* and *BlockStart* in ascending order.
- 4) Before running the query, you must set:
[sc.setJobDescription\(f"A1-P2-TrafficJoin-{student_auth_code}"\)](#)

Students must include:

- the formatted physical plan using [explain\(mode="formatted"\);](#)
- Spark UI screenshots showing the job description, any Exchange, and the relevant shuffle-read, shuffle-write values.
- a short explanation of why data movement was or was not required.

2.6 Exploratory analysis

Produce two suitable plots exploring relationships among parking activity, duration, violation rate, or traffic volume.

For each plot:

- 1) State the question being investigated;
- 2) **Display** the first five rows and the row count of the aggregated dataset used to create the plot
- 3) Select the appropriate plot type and briefly justify your choice.
- 4) Explain the observed pattern shown by the plot.
- 5) identify one limitation of the interpretation.

Part 3: Query Optimisation (25%)

Assume that the following monthly parking report will be generated regularly. A correct query is required first, but it should also execute efficiently on the full dataset. In this section, implement the report, inspect how Spark executes it, and evaluate the optimization.

Monthly parking report

For each month, identify the five areas with the highest number of peak-hour parking sessions among areas whose peak-hour violation rate is greater than the monthly violation benchmark.

- Area Violation Rate = (Peak-hour sessions where *inViolation* is true) / (Peak-hour sessions with a valid *inViolation* value) * 100
- Monthly Violation Benchmark = (Peak-hour sessions where *InViolation* is true across all areas in the month) / (Peak-hour sessions with a valid *InViolation* value across all areas in the same month) * 100

Display:

- month;
- area id;
- area name;
- park session count;
- violation count;
- violation rate;
- monthly rank.

Report the number of months and total result rows, Display the complete result ordered by Month and Monthly Rank in ascending order.

3.1 DataFrame API and Spark SQL

Implement the monthly report independently using:

1. the PySpark DataFrame API;
2. Spark SQL

Both implementations must begin with the same prepared input data. Do not create the SQL result by registering the completed DataFrame API result as a temporary view.

Complete the following:

1. Report the row count returned by each implementation
2. Confirm that the results are identical using `exceptAll()` in both directions, and display both mismatch counts.
3. For each implementation, **display** the first 20 result rows using the following column order:
 - a. Month
 - b. Areald
 - c. AreaName
 - d. PeakSessionCount
 - e. ViolationCount
 - f. ViolationRate
 - g. MonthlyViolationBenchmark
 - h. MonthlyRank
4. Order the displayed results by Month and MonthlyRank in ascending order.
5. Inspect the physical plan for each implementation, and briefly compare the operations. Briefly explain whether Spark selected different execution strategies, and why?

3.2 Baseline plan and Spark UI analysis

Select either SQL or DataFrame API from Section 3.1 as a baseline. Clearly state which implementation you selected.

Before running the baseline query, set:

`sc.setJobDescription(f"A1-P3-2-Baseline-{student_auth_code}")`

Execute an action that materialises the complete final result and record the wall-clock **execution time**. Use the same action later when evaluating the optimised query.

Submit and include:

- the formatted physical plan, you may refer to the plan already printed in Section 3.1 rather than printing it again.
- Spark UI screenshots clearly showing: the job description, the SQL/DAG, the relevant stage details, relevant shuffle-read, shuffle-write figures, where applicable.
- the measured execution time;

In no more than 300 words:

1. Identify the most expensive stages or operations.
2. Support the explanation using numbers visible in your screenshots.
3. Suggest an improvement strategy.

3.3 Evaluate one optimisation

Implement one optimisation based on the evidence and reasoning presented in Section 3.2. Briefly state what you changed and why you expect it to improve execution.

Before executing it, set:

`sc.setJobDescription(f"A1-P3-3-Optimised-{student_auth_code}")`

Then:

1. Run the baseline and revised queries under the same conditions;
2. Confirm that the optimised query produces exactly the same result as the baseline
3. Record the execution time and collect the corresponding Spark UI evidence.
4. Decide whether the optimisation should be retained, justify your decision.
5. A technically reasonable improvement and evidence-based evaluation are required for full marks.

Part 4: Interview (30%)

To demonstrate your technical understanding of the submitted work, you may choose to attend a 10–15-minute live technical interview with the teaching team. You shall pay attention to the unit announcement around the submission deadline and book your interview within one week after your submission.

- You must share your screen and demonstrate your application
- If asked, execute the code in real time to demonstrate application functionality.
- You shall adequately explain your own code (regardless of whether GenAI is used for code generation) and demonstrate a good understanding of your data

and application. **The assessment is based on your competency and understanding, regardless of the source code status.**

Note:

- You are responsible for checking your device, screen-sharing functionality and internet connection before the interview.
- Participation in the Assignment 1 interview is optional. However, students who do not complete the interview will receive zero for Part 4 and can therefore receive a maximum overall assignment mark of 70%.

Submission

Submit your final assignment solution via Moodle. You must submit the following files. Both files are required for marking; failure to submit either file may incur a penalty of up to 10%.

1. Your Jupyter notebook file (**e.g., A1_authcate.ipynb**).
2. **A PDF file** saved from jupyter notebook with **ALL** output following the file naming format as follows: **A1_authcate.pdf**
3. Required screenshots in a zip file. Please note the images **should be added to your Jupyter notebook**, this file is required just in case we can't see images from your jupyter notebook.

Important: All requested results, validation counts, plots and Spark UI evidence must be visible in both the submitted notebook and PDF. Code without its corresponding output may lose mark, unless otherwise specified.

Note that all submitted files will be scanned using plagiarism detection software. The highest similarity score among students may be asked to prove the originality of the work.

Assignment Marking Rubric

For each task individually, you'll be marked based on the quality of your work on a 3-level scale (0%, 50% and 100%).

- 0%: No attempt or incorrect answer with poor attempt;
- 50%: Partial mark for a good attempt but incorrect result;
- 100%: Full mark for correct attempt.

In your submission, the Jupyter notebook file should contain the **code and its output**. It should follow *programming standards, code readability, and code organisation*. Please find the PEP 8 -- Style Guide for Python Code for your reference. Here is the link: <https://peps.python.org/pep-0008/> **A penalty of up to 10% applies if your code is hard to understand due to insufficient comments.**

Late submissions

Late Assignments or extensions will not be accepted unless you submit a special consideration form. ALL Special Consideration, including within the semester, is now to be submitted centrally. This means that students **MUST** submit an online Special Consideration form via Monash Connect. For more details, please refer to the **Unit Information** section in Moodle.

A late submission is subject to a 5% penalty per day, including weekends. The cut-off date is 7 days after the due date. No submission (i.e. 0 mark) will be accepted after the cut-off date unless you have a special consideration.

If you submit the wrong file and ask for the submission to be reopened, the new resubmission timestamp will be used to calculate the late penalty.

Mark Release and Review

- Mark will be released within 10 business days after the submission deadline.
- Reviews and disputes regarding the mark will be accepted a maximum of 7 days after the release date (including weekends).

Other Information

Where to get help

You can ask questions about the assignment in the Assignments section in the Ed Forum accessible on the unit's Moodle Forum page. This is the preferred venue for assignment clarification-type questions. **Students are expected to check the Ed Forum regularly, as responses from the teaching staff are considered official and may constitute amendments or additions to the assignment specification. Clarifications, updates, and additional information will be posted there to help avoid confusion when completing the assignment. Failure to check the forum will not be accepted as a reason for misunderstanding the assignment requirements.**

Also, you can attend scheduled consultation sessions if the problem and confusion are still unresolved.

Plagiarism and collusion

Plagiarism and collusion are serious academic offences at Monash University. Students must not share their work with any other students. Students should consult the policy linked below for more information.

<https://www.monash.edu/students/academic/policies/academic-integrity>

See also the video linked on the Moodle page under the Assignment block.

Students involved in collusion or plagiarism will be subject to disciplinary penalties, which can include:

- The work is not being assessed
- A zero grade for the unit

- Suspension from the University
- Exclusion from the University

Generative AI Statement

As per the University's [policy](#) on the guidelines and practices pertaining to the usage of Generative AI:

AI & Generative AI tools may be used SELECTIVELY within this assessment.

Where used, AI must be used responsibly, clearly documented and appropriately acknowledged (see Learn HQ).

Any work submitted for a mark must:

- 1) Represent a sincere demonstration of your human efforts, skills and subject knowledge that you will be accountable for.
- 2) Adhere to the AI use guidelines set for the assessment task.
- 3) Reflect the University's commitment to academic integrity and ethical behaviour.

Inappropriate AI use and/or AI use without acknowledgement will be considered a breach of academic integrity.

The teaching team encourages students to apply their own critical thinking and reasoning skills when working on assessments, with assistance from GenAI. Generative AI tools may produce inaccurate content, which could negatively impact students' comprehension of big data topics.

Data source acknowledgement:

The dataset is a remix based on several real-world datasets.